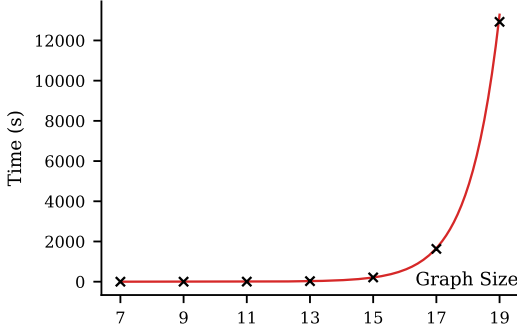


## A Computational Scalability Analysis

As discussed in the main text, current methods often rely on the exhaustive enumeration of the evader’s action space. This requires pre-calculating all valid simple paths from the starting position to the exits within a finite time horizon  $T$ . While feasible for small maps, this approach faces severe scalability issues.

To quantify this limitation, we measured the time cost required to enumerate simple paths from the center of a grid ( $N \times N$ ) to the corners, setting the time horizon  $T = N$ . Figure 5 illustrates the relationship between grid size and computation time. The results demonstrate an exponential explosion in computational overhead. Specifically, the runtime becomes prohibitively high once the grid size exceeds  $20 \times 20$ . This trend confirms that reliance on global action space traversal constitutes a fundamental scalability barrier, necessitating the data-driven approach proposed in this work.



**Figure 5: Time cost for enumerating evader simple paths from the grid center to four corners. The solid line represents the exponential trend, fitted via linear regression on the logarithmic scale. The rapid growth underscores the intractability of path-based methods on large-scale graphs.**

## B User-controllable parameters

The user-controllable parameters are shown in Table 7. In our platform, users configure a graph-based pursuit-evasion game by providing a **base graph structure** and specifying game-specific settings through the `GameSettings` class. The base graph can be loaded from a NetworkX gpickle file, which defines the topology of the environment. We also provide a graph generation utility that allows users to create grid-based graphs through parameters such as `grid_width` and `grid_height`, which define the grid dimensions; `axis_edge_prob`, which controls the probability of retaining horizontal and vertical edges; and `diag_edge_prob`, which determines the probability of adding diagonal edges. These parameters enable users to generate diverse graph topologies ranging from sparse to fully-connected grid structures.

Users can further customize the graph structure through the `metadata` parameter. By specifying an `adjacency_matrix` along with corresponding `node_ids`, users can override the base graph topology with a completely custom structure. The adjacency matrix can be either **symmetric** (for undirected graphs) or **asymmetric** (for directed graphs), with each entry representing the edge weight between nodes. This flexible design enables users to model

**Table 7: The parameters that users can control.**

|                            | Graph Generation                                              | Graph Customization                                                             |
|----------------------------|---------------------------------------------------------------|---------------------------------------------------------------------------------|
| underlying graph structure | grid_width<br>grid_height<br>axis_edge_prob<br>diag_edge_prob | adjacency_matrix<br>node_ids<br>use_weighted_graph                              |
| agent number and position  |                                                               | time_horizon<br>evader_init<br>pursuer_init<br>exit_nodes<br>graph_gpickle_path |

diverse scenarios, from simple grid-like environments to complex directed weighted networks. Additionally, the `use_weighted_graph` parameter controls whether edge weights are utilized in the game dynamics; when set to `False`, all edges default to unit weight.

For the **agent configuration and game parameters**, users control the number and initial positions of agents through `evader_init` and `pursuer_init`, which specify the starting node indices for each type of agent. The `exit_nodes` parameter defines the set of goal locations that evaders aim to reach, while `time_horizon` sets the maximum simulation duration. This parameterization allows users to construct a wide range of pursuit-evasion scenarios tailored to specific research questions or applications.

## C Faster wall-clock Convergence

Our platform incorporates several architectural and implementation enhancements that contribute to faster wall-clock convergence times compared to the original implementations. These improvements span environment simulation, configuration management, data processing pipelines, and computational resource utilization.

**Standardized Environment Interface.** We have adopted the Gymnasium framework as the standard interface for all game environments, replacing the custom class-based implementations found in the original papers. The original code (e.g., `build_game` in `game_config.py`) constructs game instances through deeply nested function calls with numerous embedded parameters, resulting in redundant data copying and inefficient state transitions. In contrast, our `UNSGEnv` class implements the standard Gymnasium API with optimized step and reset methods, eliminating unnecessary data transformations between the environment and agents. This standardization not only accelerates simulation processes but also ensures compatibility with modern reinforcement learning libraries.

**Efficient Graph Representation and Loading.** The original implementation rebuilds graph structures during environment initialization, often involving multiple graph transformations and node embedding computations for each experiment run. Our platform leverages NetworkX’s gpickle serialization format to pre-compute and cache graph structures. Graphs are loaded once via `pickle.load()` and wrapped in a `GameSettings` object, which efficiently manages edge weights and neighbor mappings through pre-built dictionaries. This approach eliminates redundant graph construction overhead across training iterations.

**Streamlined Configuration Management.** We employ YAML-based configuration files combined with command-line overrides (via `load_yaml_config`), replacing the hardcoded parameter specifications scattered throughout the original codebase. The original implementation embeds configuration parameters directly in execution scripts, making parameter tuning and reproducibility challenging. Our configuration system centralizes all hyperparameters, enables programmatic parameter sweeps, and automatically saves experiment configurations via `save_experiment_config`, facilitating efficient experimentation and result tracking.

**Optimized Data Processing Pipeline.** We have implemented various data processing optimizations throughout the training pipeline. The original code frequently converts between Python lists, numpy arrays, and PyTorch tensors, incurring significant overhead during high-frequency operations such as state observation extraction and action sampling. Our implementation maintains consistent data representations using numpy arrays for observations (via `get_current_obs` returning structured numpy arrays) and PyTorch tensors for neural network computations, minimizing type conversion costs. Additionally, we have optimized the dimension computation functions (`compute_input_dim`, `compute_action_dim`) to perform calculations once during initialization rather than repeatedly during training.

**Modular Component Design.** Our platform separates concerns through well-defined abstractions: `env_builder` for environment instantiation, `PPOAgent` for policy networks, `PPOAlgorithm` for learning updates, and specialized runners (`AttackerPathRunner`, `DefenderPretrainPsroRunner`) for training orchestration. This modularity contrasts with the monolithic `PSRO_Oracle` class in the original implementation, which tightly couples environment interactions, policy updates, and meta-strategy computation. Our design reduces redundant computations by enabling component-level caching and reuse across PSRO iterations.

**Vectorized Environment Support.** Unlike the original sequential episode sampling, our platform supports vectorized environments through the `vec_envs` parameter, enabling parallel data collection across multiple environment instances. This parallelization capability substantially reduces wall-clock training time, particularly during the pretrain phase where large numbers of episodes (pretrain\_episodes\_per\_task) are required.

It is important to emphasize that these enhancements primarily accelerate wall-clock convergence without altering the fundamental algorithmic behavior. The number of training iterations required for convergence remains approximately consistent with the original implementations. For instance, if the original PSRO algorithm requires  $10^4$  episode samples to converge, our platform’s reproduced version similarly requires a comparable number of training iterations. This consistency validates that our optimizations preserve algorithmic fidelity while delivering substantial wall-clock speedups through improved implementation efficiency.

## D Description of Experimental Settings

The graph structures for the  $5 \times 5$  and  $7 \times 7$  grids are illustrated in Figure 6 and 7.

The reason why the left side of Figure 7 reaches a probability of 0.5 under the Nash equilibrium is explained as follows:

**Exit Node Accessibility:** Among the available exit nodes, two (located at the bottom-left and bottom-right) require more than three steps for the evader to reach. However, the pursuers can reach these exits in exactly three steps. As a result, a rational evader will avoid these exits, since choosing them would guarantee capture.

**Viable Exit Options:** This constraint leaves only two viable exit nodes for the evader. Only the pursuer positioned at the top-left corner is able to reach either of these exits within three steps. Furthermore, both exits present equal strategic value for both the evader and the pursuer.

**Probability Calculation:** Rational decision-making leads the evader to choose between these two viable exits with equal probability (i.e.,  $1/2$  for each). For each exit, the probability of being caught depends on whether the pursuer also chooses the same exit, which is again  $1/2$ . Thus, the total probability is determined as:  $2 \text{ exits} \times (1/2 \text{ chance of the evader choosing each exit}) \times (1/2 \text{ chance of the pursuer selecting the same exit}) = 0.5$ .

A similar reasoning process applies to the scenario on the left side of Figure 6. Under the Nash equilibrium, the probability that the evader is captured is also 0.5.

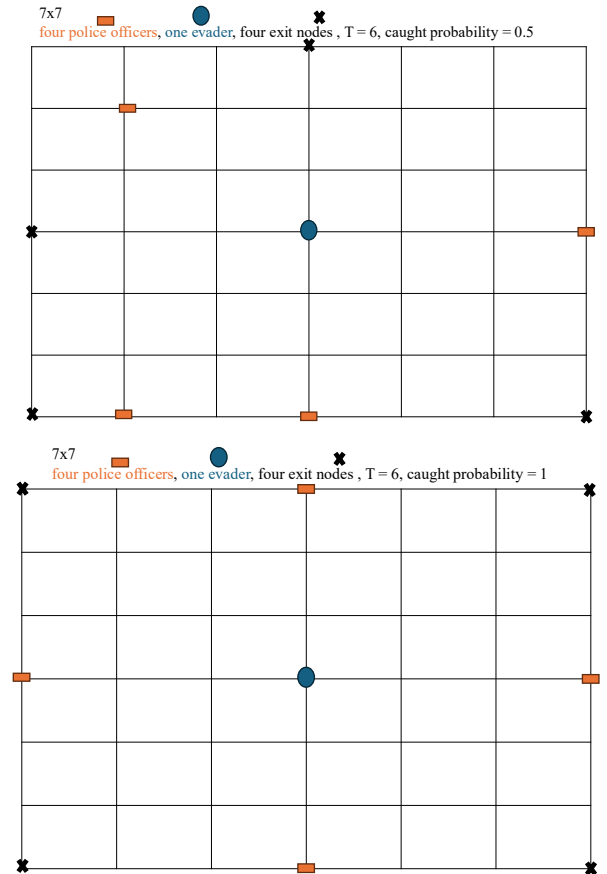


Figure 6:  $7 \times 7$  grid graph settings with the evader caught probability of 0.5 (left) or 1 (right).

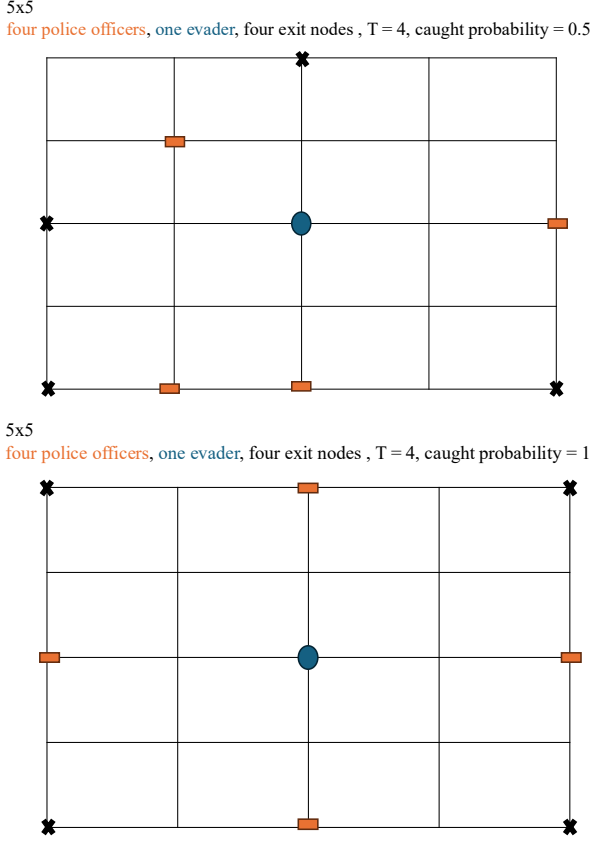


Figure 7:  $5 \times 5$  grid graph settings with the evader caught probability of 0.5 (left) or 1 (right).

## E Detailed Visualizations of Graph Structures

To provide further details on the experimental setup, we present the specific network topologies used in our evaluation. Figure 8 illustrates the eight distinct graph structures employed. These include both structured synthetic grids and irregular real-world road networks.

## F Visualization Evaluation

This section presents the Visualization evaluation method provided by GraphChase. Figure 9 captures snapshots from a recorded pursuit-evasion episode, illustrating the position changes of the pursuers and the evader at each time step. Step 1 shows the initial positions of the evader and the pursuers. Each subsequent step reflects one move by each side. In Step 6, the evader and pursuer overlap, indicating that the evader has been captured by a pursuer.

## G Usage Instructions for GraphChase

The following steps outline the process for setting up and utilizing the GraphChase platform:

### G.1 Cloning the Repository

To begin, clone the GraphChase repository from GitHub and navigate to the project directory:

```
git clone \
https://github.com/GraphChase/GraphChasePlatform.git
cd GraphChasePlatform
```

### G.2 Installing Dependencies

Install the necessary dependencies, including PyTorch, NetworkX, Gymnasium, and other required packages. Refer to the requirements.txt file for the complete list of dependencies.

### G.3 Preparing Graph Structures

Our platform uses NetworkX gpickle format for graph storage. Pre-generated graph files are available in the new\_version/graph/custom\_graph/ directory. To create custom graphs, use the graph generation utility:

```
python -m new_version.graph.generate_custom_graph \
--grid_width 7 --grid_height 7 \
--axis_edge_prob 1.0 --diag_edge_prob 0.0 \
--output_path custom_graph.gpickle
```

### G.4 Running an Algorithm

To run a specific algorithm, such as Pretrain-PSRO, perform the following steps:

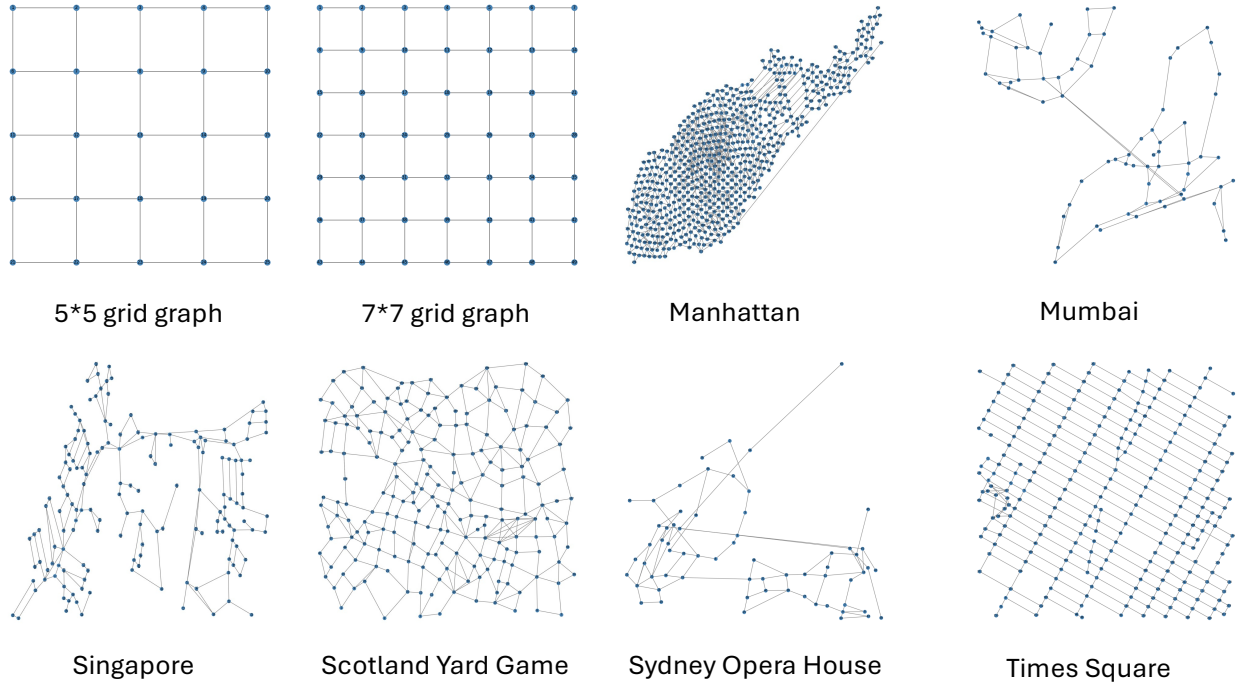
- (1) **Select or Create Configuration File:** Choose an appropriate YAML configuration file from the new\_version/solver\_cfgs/ directory (e.g., pretrain\_psro\_cfgs\_7\_7\_5.yaml), or create a new one based on existing templates. The configuration file specifies:
  - **Graph arguments:** Path to the gpickle file (graph\_gpickle\_path), agent initial positions (attacker\_init, defender\_init), exit nodes (exit\_nodes), and time horizon
  - **Graph embedding arguments:** Whether to use graph embeddings, embedding parameters, and paths to pre-computed embeddings
  - **Algorithm hyperparameters:** PSRO iterations, training batch sizes, PPO learning rates, and other algorithm-specific parameters
  - **Experiment settings:** Random seed, GPU configuration, output paths, and logging preferences
- (2) **Customize Parameters (Optional):** To override specific configuration values without modifying the YAML file, use command-line arguments with the -set flag. For example, to change the random seed:

```
--set seed=3407
```

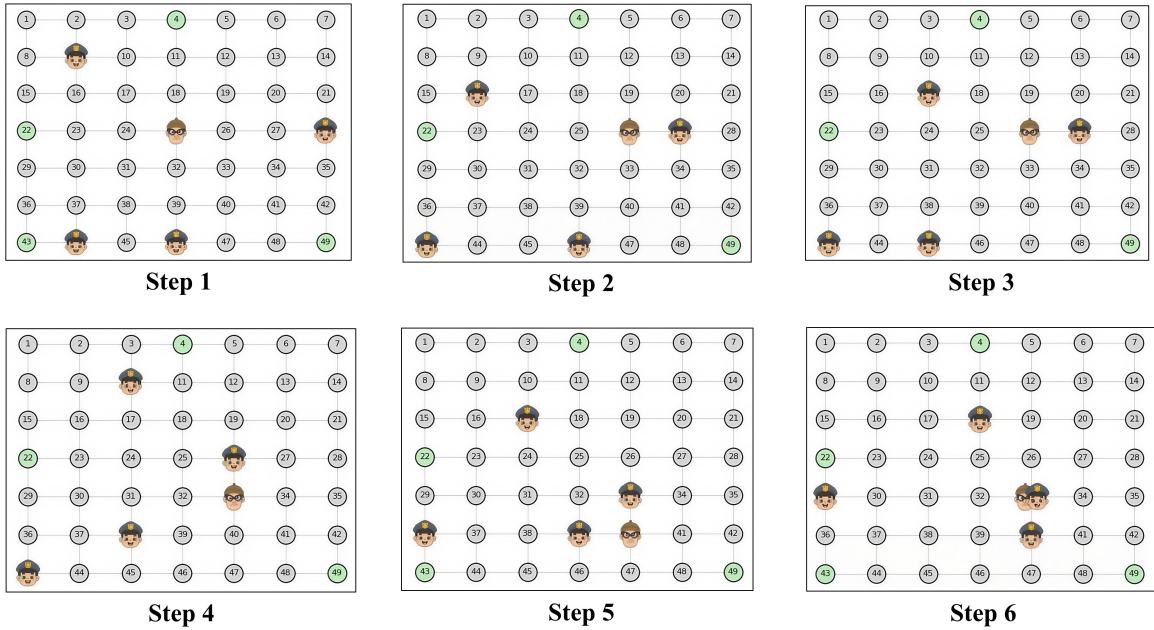
- (3) **Run the Algorithm:** Execute the algorithm using one of the following methods:

#### Method 1 - Direct Python Execution:

```
python -m new_version.scripts.run_pretrain_psro \
--config new_version/solver_cfgs/\
pretrain_psro_cfgs_7_7_5.yaml \
--set seed=3407
```



**Figure 8: Visualization of the eight graph structures employed in the experiments. The dataset comprises synthetic grid graphs ( $5 \times 5$ ,  $7 \times 7$ ) and real-world topologies.**



**Figure 9: Snapshots from a recorded pursuit-evasion episode.**

**Method 2 - Shell Script for Multi-seed Experiments:**

```
bash run_pretrainpsro_multiseed.sh
```

This script automates the process of running experiments across multiple random seeds, particularly useful for statistical validation and cluster computing environments.

All algorithms utilize the same YAML-based configuration system, enabling consistent parameterization and reproducible experimentation across different solution methods.

**H Hyperparameter Configuration**

The hyperparameter configurations for all implemented algorithms are detailed in Tables 8, 9, 10, 11, and 12. These configurations are based on the experimental setting with a  $7 \times 7$  grid graph. For additional experimental configurations, including different graph sizes, agent numbers, and real-world map scenarios, pre-configured YAML files are available in the `new_version/solver_cfgs/` directory of our GitHub repository.

**Table 8: Hyperparameters for Pretrain-PSRO algorithm (7×7 grid, captured  $prob = 0.5$ ).**

| Game Configuration           |           |                       |          |
|------------------------------|-----------|-----------------------|----------|
| time_horizon                 | 7         | seed                  | 77       |
| use_weighted_graph           | false     | device_id             | 0        |
| Graph Embedding Parameters   |           |                       |          |
| graph_embeddings             | true      | emb_size              | 16       |
| node_information_type        | all       | normalize_info        | true     |
| similarity                   | cosine    | line_order            | all      |
| epochs                       | 200       | batch_size            | 32       |
| neg_samples                  | 5         | lr                    | 0.025    |
| PSRO Parameters              |           |                       |          |
| num_psro_iteration           | 20        | eval_episodes         | 1000     |
| rollouts_per_attacker_action | 10000     | vec_envs              | 16       |
| train_defender_batches       | 630       | episodes_per_batch    | 8        |
| Pretraining Parameters       |           |                       |          |
| pretrain_iterations          | 40        | pretrain_tasks        | 30       |
| pretrain_episodes_per_task   | 20        | load_pretrained_model | false    |
| PPO Hyperparameters          |           |                       |          |
| ppo_actor_lr                 | 0.001     | ppo_critic_lr         | 0.003    |
| ppo_gamma                    | 0.99      | ppo_lambda            | 0.95     |
| ppo_clip                     | 0.2       | ppo_epochs            | 10       |
| ppo_batch_size               | 32        | ppo_hidden_dim        | 128      |
| entropy_coef                 | 0.01      |                       |          |
| Attacker Settings            |           |                       |          |
| action_type                  | exit_node | strategy_type         | mix      |
| attacker_path_type           | shortest  | reward_mode           | win_rate |

**Table 9: Hyperparameters for CFR-Mix algorithm (7×7 grid, captured  $prob = 0.5$ ).**

| Game Configuration           |       |                      |        |
|------------------------------|-------|----------------------|--------|
| time_horizon                 | 7     | seed                 | 77     |
| use_weighted_graph           | false | device_id            | 0      |
| Network Architecture         |       |                      |        |
| network_dim                  | 32    |                      |        |
| Training Parameters          |       |                      |        |
| iteration                    | 10000 | train_epoch          | 1000   |
| sample_number                | 20    | action_number        | 1000   |
| Attacker Regret Network      |       |                      |        |
| attacker_regret_batch_size   | 32    | attacker_regret_lr   | 0.0015 |
| Defender Regret Network      |       |                      |        |
| defender_regret_batch_size   | 512   | defender_regret_lr   | 0.0015 |
| Defender Strategy Network    |       |                      |        |
| defender_strategy_batch_size | 32    | defender_strategy_lr | 0.0015 |

**Table 10: Hyperparameters for NSGZero algorithm (7×7 grid, captured  $prob = 0.5$ ).**

| Game Configuration     |       |                      |          |
|------------------------|-------|----------------------|----------|
| time_horizon           | 7     | seed                 | 77       |
| use_weighted_graph     | false | device_id            | 0        |
| save_model             | true  | use_tensorboard      | false    |
| Network Architecture   |       |                      |          |
| embedding_dim          | 16    | hidden_dim           | 256      |
| Training Parameters    |       |                      |          |
| max_episodes           | 6000  | batch_size           | 128      |
| buffer_size            | 50000 | lr                   | 0.0005   |
| num_workers            | 4     | debug_single_process | false    |
| Training Schedule      |       |                      |          |
| train_every            | 16    | train_from           | 128      |
| test_every             | 250   | test_nepisodes       | 50       |
| save_every             | 500   | log_every            | 500      |
| MCTS Parameters        |       |                      |          |
| num_sims               | 150   | bias                 | 0.5      |
| cpuct                  | 0.3   | temp                 | 0.5      |
| gamma                  | 1.0   | reward_mode          | win_rate |
| Attacker Configuration |       |                      |          |
| att_type               | nfsp  | ban_capacity         | 500      |
| cache_capacity         | 20    | br_rate              | 0.2      |

**Table 11: Hyperparameters for NSG-NFSP algorithm (7×7 grid, captured  $prob = 0.5$ ).**

| Game Configuration          |         |                     |          |
|-----------------------------|---------|---------------------|----------|
| time_horizon                | 7       | seed                | 0        |
| use_weighted_graph          | false   | device_id           | 0        |
| Network Architecture        |         |                     |          |
| embedding_size              | 32      | hidden_size         | 64       |
| relevant_v_size             | 64      | if_naivedrrn        | false    |
| seq_mode                    | cnn     |                     |          |
| Replay Buffer Configuration |         |                     |          |
| br_buffer_capacity          | 500000  | avg_buffer_capacity | 10000000 |
| Learning Parameters         |         |                     |          |
| br_lr                       | 0.0001  | avg_lr              | 0.0001   |
| br_batch_size               | 128     | avg_batch_size      | 256      |
| Exploration Parameters      |         |                     |          |
| d_expl                      | 0.0     | a_expl              | 0.1      |
| br_prob                     | 0.1     |                     |          |
| Training Schedule           |         |                     |          |
| max_episodes                | 1200000 | train_br_freq       | 4        |
| train_avg_freq              | 32      | check_freq          | 100000   |
| check_from                  | 100000  | display_freq        | 10000    |
| min_to_train                | 1000    | exact_br            | false    |
| Agent Configuration         |         |                     |          |
| defender_rl_mode            | drnn    | defender_sl_mode    | drnn     |
| attacker_mode               | bandit  | reward_mode         | win_rate |

**Table 12: Hyperparameters for GRASPER-MAPPO algorithm (7×7 grid, captured  $prob = 0.5$ ).**

| Game Configuration            |            |                        |          |
|-------------------------------|------------|------------------------|----------|
| time_horizon                  | 7          | seed                   | 77       |
| use_weighted_graph            | false      | device_id              | 0        |
| graph_type                    | Grid_Graph | edge_probability       | 0.8      |
| row                           | 7          | column                 | 7        |
| Graph Embedding Configuration |            |                        |          |
| use_node_emb                  | true       | load_graph_emb_model   | true     |
| max_epoch                     | 200        |                        |          |
| Game Pool Configuration       |            |                        |          |
| load_game_pool_file           | true       | pool_size              | 1000     |
| min_num_defender              | 4          | max_num_defender       | 4        |
| min_num_exit                  | 4          | max_num_exit           | 5        |
| min_attacker_pth_len          | 5          |                        |          |
| PSRO Parameters               |            |                        |          |
| num_psro_iteration            | 20         | train_defender_batches | 1200     |
| episodes_per_batch            | 32         | reward_mode            | win_rate |
| MAPPO Hyperparameters         |            |                        |          |
| ppo_epoch                     | 8          | minibatch_size         | 64       |
| use_gae                       | false      | use_act_supervisor     | true     |
| Training Schedule             |            |                        |          |
| num_iterations                | 20000      | save_every             | 1000     |